CURSO BACKEND JAVA — AULA 01 (GUIA PRÁTICO)

Fundamentos de Java, Orientação a Objetos e Ecossistema

Esta aula aborda os pilares fundamentais da linguagem Java e da Programação Orientada a Objetos (POO), focando na aplicação prática em sistemas corporativos. Entenderemos desde a manipulação de tipos primitivos e tipos montados até a estruturação de classes, métodos, modificadores de acesso, além de uma introdução direta às ferramentas do ecossistema real (Maven, Spring Boot e Quarkus).

1. SISTEMA DE TIPOS: PRIMITIVOS VS. TIPOS MONTADOS (WRAPPERS E OBJETOS)

O Java possui um sistema de tipos estático e forte. Os dados são divididos em duas categorias essenciais na memória: **Primitivos** (guardados diretamente na memória Stack por valor) e **Tipos Montados/Referência** (alocados na memória Heap com referências na Stack).

Tipo	Categoria	Tamanho / Formato	Uso Prático no Backend e Armadilhas
`int` / `long`	Primitivo	32-bit / 64-bit	Operações matemáticas puras e contadores. Não aceita `null`.
`double` / `float`	Primitivo	Ponto Flutuante IEEE 754	**NUNCA use para dinheiro!** Ocorre perda de precisão binária.
`boolean` / `char`	Primitivo	1-bit (lógico) / 16-bit (UTF-16)	Flags lógicas simples e caracteres individuais.
`Integer` / `Long`	Wrapper (Objeto)	Classe envelopadora	Permite representar o estado **ausente (null)** vindo do banco de dados.
`String`	Objeto Especial	Sequência Imutável de Chars	Texto no geral. Usar métodos de comparação com cuidado (`.equals()`).
`BigDecimal`	Objeto de Negócio	Precisão Arbitrária Exacta	**Obrigatório para finanças, saldos e valores monetários.**

1.1. Por que a diferença entre Primitivo e Wrapper importa no Backend?

Nos bancos de dados relacionais, uma coluna como `idade` ou `valor_desconto` pode possuir valor nulo (`NULL`). Se no seu código Java a propriedade for declarada como primitivo `int idade;`, o Java atribuirá o valor padrão `0`, mascarando a ausência do dado. Declarando como `Integer idade;`, a variável pode receber `null`, mantendo a integridade com o banco.

```
      // Exemplo Prático: Comparação incorreta vs correta de Strings e Wrappers String token1 = new String("AUTH123"); String token2 = new String("AUTH123"); boolean comparaPonteiro = (token1 == token2); // FALSE: compara endereço de memória! boolean comparaConteudo = token1.equals(token2); // TRUE: compara os caracteres reais. Long idA = 1000L; Long idB = 1000L; boolean idIgual = idA.equals(idB); // Sempre use .equals() para comparar objetos Wrappers!
```

1.2. Literais e Pós-fixos Obrigatórios

Ao declarar valores numéricos no código, o compilador interpreta números inteiros como `int` e decimais como `double` por padrão. Para especificar tipos maiores ou decimais de precisão simples, utiliza-se sufixos de literais:

- `long limite = 5000000000L;` (Sufixo `L` indica que o número supera o limite do `int`).
- `float taxa = 3.14f;` (Sufixo `f` evita o erro de conversão implícita de `double` para `float`).
- `BigDecimal valor = new BigDecimal("150.75");` (Sempre passe a String no construtor do BigDecimal).

2. ESTRUTURA DE CLASSES, PROPRIEDADES E ASSINATURA DE MÉTODOS

2.1. O Molde (Classe) e o Estado (Propriedades / Atributos)

Uma **Classe** é o projeto estrutural (molde) que define o que um elemento do domínio possui (atributos) e o que ele faz (métodos). O estado de um objeto reside no valor de suas propriedades em um dado momento.

```
package com.br.empresa.domain; import java.math.BigDecimal; public class ContaBancaria { // Propriedades (Atributos de Instância) private String numeroConta; private String titular; private BigDecimal saldo; // Construtor: invocado no momento da criação para inicializar o objeto public ContaBancaria(String numeroConta, String titular, BigDecimal saldoInicial) { this.numeroConta = numeroConta; this.titular = titular; this.saldo = (saldoInicial != null) ? saldoInicial : BigDecimal.ZERO; } }
```

2.2. Anatomia da Assinatura de um Método

A **assinatura de um método** é o contrato público que declara como outros componentes interagem com ele. Ela é composta por: *Modificador de Acesso* + *Modificador de Estado* + *Tipo de Retorno* + *Nome do Método* + *(Parâmetros)* + *Exceções Declaradas*.

```
// Estrutura Completa da Assinatura de um Método public final BigDecimal depositar(BigDecimal valorOperacao) throws IllegalArgumentException { if (valorOperacao == null || valorOperacao.compareTo(BigDecimal.ZERO) <= 0) { throw new IllegalArgumentException("O valor de depósito deve ser estritamente positivo."); } this.saldo = this.saldo.add(valorOperacao); return this.saldo; // Retorna o saldo atualizado após o depósito }
```

- **public**: Visibilidade total por qualquer outra classe do projeto.
- **BigDecimal**: O tipo de dado que a função devolve para quem a chamou. (Se não devolver nada, usa-se `void`).
- **depositar**: Nome do método, utilizando a convenção *camelCase* iniciando com verbo.
- **(BigDecimal valorOperacao)**: Parâmetro obrigatório recebido de fora para execução do processamento.

2.3. Modificadores de Acesso (`public` vs `private`) e Encapsulamento

O princípio do **Encapsulamento** prevê que os dados de uma classe não devem ser modificados diretamente por código externo sem passar por validações.

- `private`: Restringe o acesso ao atributo/método exclusivamente à própria classe. Impede reatribuições diretas e ilegais de valores externos (ex: `conta.saldo = new BigDecimal("-100");` é bloqueado no compilação).
- `public`: Expõe o método para o mundo externo. Métodos públicos atuam como portas de entrada seguras e controladas para alterar os atributos privados.

3. INSTANCIAÇÃO, CONSTRUTORES E A PALAVRA-CHAVE `STATIC`

3.1. Instanciação com a Palavra-Chave `new`

Instanciar uma classe significa materializá-la na memória em tempo de execução, transformando a definição estática em um **Objeto** operável e isolado. A palavra-chave `new` aloca espaço no Heap, inicializa os atributos e invoca o construtor.

```
      // Instanciando dois objetos independentes da mesma classe ContaBancaria ContaBancaria
contaMaria = new ContaBancaria("1001-X", "Maria Silva", new BigDecimal("500.00")); ContaBancaria
contaJoao = new ContaBancaria("1002-Y", "João Souza", new BigDecimal("1500.00"));
contaMaria.depositar(new BigDecimal("200.00")); // Altera APENAS o saldo de contaMaria!
```

3.2. Membros da Classe vs. Membros da Instância (`static`)

A palavra-chave `static` altera completamente o escopo de um atributo ou método. Quando algo é marcado como `static`, ele pertence à **Classe como um todo** e não a cada instância individual. Todos os objetos criados compartilham exatamente a mesma variável estática na memória.

```
      public class ValidadorDocumentoUtil { // Atributo estático: compartilhado por toda a
aplicação na memória public static final int TAMANHO_CPF = 11; // Método estático: pode ser
chamado DIRETO pela Classe, sem dar 'new'! public static boolean isCpfValido(String cpf) { if (cpf
== null || cpf.length() != TAMANHO_CPF) { return false; } return cpf.matches("\\d{11}"); // Regra
simplificada de validação } } // Chamada direta sem necessidade de instanciação (sem o uso de
'new'): boolean valido = ValidadorDocumentoUtil.isCpfValido("12345678901");
```

3.3. Quando Usar e Quando Evitar o `static`

Cenário	Uso do `static`	Justificativa Técnica
Funções Utilitárias Puras (Matemática, Validações)	**RECOMENDADO**	Recebe parâmetros de entrada e retorna resultado sem depender de estado interno.
Constantes Globais do Sistema (`public static final`)	**RECOMENDADO**	Evita criar múltiplas cópias de um mesmo valor fixo (ex: URL base, limite) na Heap.
Injeção de Serviços / Controllers em Spring	**PROIBIDO / EVITAR**	O `static` impede que o container de Injeção de Dependência gerencie o ciclo de vida.
Variáveis mutáveis em servidores Web (Multithread)	**PERIGOSO**	Gera *Race Condition*. A requisição do Usuário A pode sobrescrever dados do Usuário B!

4. ECOSSISTEMA ENTERPRISE: MAVEN, SPRING BOOT E QUARKUS

4.1. Apache Maven (Gerenciador de Build e Dependências)

Sem o Maven, você teria que baixar arquivos `.jar` das bibliotecas manualmente e adicioná-los ao classpath. O Maven automatiza o download de dependências transitivas e o ciclo de compilação através do arquivo central `pom.xml`.

```
<!-- Exemplo de declaração de dependência no pom.xml --> <dependency>
<groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-web</artifactId>
<version>3.2.0</version> </dependency>
```

Fases do Ciclo de Vida do Maven:

- `mvn clean`: Apaga os artefatos compilados anteriormente na pasta `target/`.
- `mvn compile`: Compila o código fonte Java em bytecode (`.class`).
- `mvn test`: Executa automaticamente os testes unitários e de integração (JUnit/Mockito).
- `mvn package`: Empacota o projeto em um arquivo executável `.jar` pronto para deploy.

4.2. Spring Boot vs. Quarkus (Frameworks de Backend)

Frameworks eliminam a necessidade de reescrever código de infraestrutura (servidores web integrados, conexões com banco de dados, roteamento de URLs).

- **Spring Boot:** O padrão da indústria Java. Baseado em Inversão de Controle (IoC) e Injeção de Dependência (DI). Ele descobre componentes anotados em tempo de execução através de *Reflection* e sobe um servidor Tomcat interno automaticamente.
- **Quarkus:** Framework moderno feito para nuvem e Kubernetes ("Supersonic Subatomic Java"). Ele executa a injeção de dependência e otimizações durante a **compilação** (build-time), resultando em tempo de inicialização de milissegundos e consumo baixíssimo de memória, além de suportar compilação nativa via GraalVM.

4.3. Exemplo Prático de API Mínima em Spring Boot vs. Quarkus

Ambos os frameworks permitem criar endpoints REST usando anotações simples que eliminam a criação manual de servidores HTTP:

```
// Endpoint REST mínimo em Spring Boot @RestController @RequestMapping("/api/saudacao")
public class SpringSaudacaoController { @GetMapping public String dizerOla() { return "Olá!
Aplicação Spring Boot em execução!"; } } // Endpoint REST equivalente no Quarkus (usando
especificação Jakarta REST / JAX-RS) @Path("/api/saudacao") public class QuarkusSaudacaoResource {
@GET @Produces(MediaType.TEXT_PLAIN) public String dizerOla() { return "Olá! Aplicação Quarkus em
execução!"; } }
```